

Eager Loading in Entity Framework Core

In this article, I will discuss **Eager Loading in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [LINQ to Entities Queries in EF Core](#). At the end of this article, you will learn the different ways to load the related entities. We will work with the same example we have worked on so far.

What do you mean by Related Entities in Entity Framework Core?

In Entity Framework Core (EF Core), related entities refer to entities that have relationships with each other, which are typically represented by navigation properties. Relationships can be one-to-one, one-to-many, or many-to-many. For a better understanding, please look at the Student entity we created in our previous article. In this case, the Branch, Address, and Course entities are related entities of the Student entity.

```
public class Student
{
    0 references
    public int StudentId { get; set; }
    0 references
    public string FirstName { get; set; }
    0 references
    public string LastName { get; set; }
    0 references
    public string Gender { get; set; }
    0 references
    public DateTime DateOfBirth { get; set; }
    0 references
    public int BranchId { get; set; }
    0 references
    public Branch Branch { get; set; }
    0 references
    public Address Address { get; set; }
    0 references
    public ICollection<Course> Courses { get; set; }
}
```

 **Related Entities of Student Entity**

Here:

- **Branch:** A Student belongs to a Branch, which makes the Branch a related entity to the Student. This is a many-to-one relationship because multiple students can belong to one branch. Each student has a foreign key (BranchId) that references the Branch they are associated with. One Branch can have many Students, but each Student belongs to only one Branch.
- **Address:** A Student has an Address, making the Address another related entity. This is a one-to-one relationship since each student can have only one address, and each address is linked to only one student. The Address table contains the foreign key (StudentId) to establish this relationship. Each Student can have one Address, which is linked to one Student.
- **Courses:** A Student can enroll in multiple Courses, making Courses a related entity in a many-to-many relationship. Many Students can enroll in multiple Courses, and many Courses can have multiple Students enrolled. The relationship is managed through an intermediate or junction table.

How many ways can we load the Related Entities in Entity Framework Core?

When loading the Student entity (i.e., retrieving data from the Students table), you can retrieve the related entities, Branch, Address, and Courses, in three different ways. They are as follows:

- **Eager Loading in EF Core:** In Eager Loading, related data (such as Branch, Address, and Courses) is loaded automatically as part of the initial query to the database. This is done using the `Include()` and `ThenInclude()` methods. The related entities are loaded in the same query when the Student entity is retrieved.
- **Lazy Loading in EF Core:** In Lazy Loading, the related data is not retrieved initially. Instead, the related entities are loaded only when we access the navigation property for the first time. EF Core makes a separate query to the database when we try to access the related property.
- **Explicit Loading in EF Core:** In Explicit Loading, related data is not loaded initially or automatically when we access the navigation property. Instead, we explicitly request the related entities to be loaded later using methods like `Entry().Reference().Load()` for single entities (e.g., Branch) or `Entry().Collection().Load()` for collections (e.g., Courses).

Note: In this article, we will discuss Eager Loading in detail, and in our next two articles, we will discuss [Lazy Loading](#) and [Explicit Loading](#) with Examples.

What is Eager Loading in Entity Framework Core?

Eager Loading in Entity Framework Core (EF Core) is a mechanism that allows related entities to be loaded alongside the main entity in a single database query using SQL JOINS. This approach is efficient in scenarios where we know in advance that we will need related data immediately, as it reduces the number of database queries by fetching all necessary data in a single query. This helps avoid the “**N+1 Query Problem**,” where multiple queries would otherwise be sent to the database to fetch each related entity one by one.

For example, if you need to retrieve a Student and load the related Branch and Address entities simultaneously, Eager Loading allows us to include these entities in the same query. This is especially useful when we expect to use the related data immediately and want to minimize the number of database round trips.

Key Points of Eager Loading:

- **Single Query:** Eager loading reduces the number of database calls by retrieving all necessary data in one query.
- **Prevents the N+1 Problem:** Fetching related data in one go prevents performance issues caused by multiple queries (the N+1 query problem).
- **Use case:** Eager Loading is ideal when you know we need the related entities along with the main entity immediately and want to minimize the number of database queries.

How Do We Implement Eager Loading in Entity Framework Core?

Eager Loading in Entity Framework Core (EF Core) is implemented using the `Include()` method. This method allows us to specify which related entities should be loaded alongside the main entity in a single query.

In scenarios where we need to load related entities at multiple levels (e.g., the related entity of a related entity), EF Core also provides the `ThenInclude()` method, which lets you eagerly load nested related entities. This feature was not available in Entity Framework 6 but is supported in EF Core. So, the following two methods we can use to load the related and nested entities in EF Core:

- **Include() Method:** Used to load the related entity (e.g., loading Branch when retrieving Student).
- **ThenInclude() Method:** Used to load further levels of related entities (e.g., loading Courses of a Student and then Subjects within those courses).

Example to Understand Eager Loading in Entity Framework Core:

Our requirement is that when loading the Student entities, we also need to load the corresponding Address entities eagerly. For a better understanding, please modify the Program class as follows. In the below code, we implement

Eager Loading using both method syntax and query syntax in Entity Framework Core, including both versions of the Include method (one using a lambda expression and the other using a string parameter). The following example code is self-explained, so please read the comment lines for a better understanding.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Initialize the database context
            using (var context = new EFCoreDbContext())
            {
                try
                {
                    // Method Syntax with Include (using lambda expression)
                    Console.WriteLine("Method Syntax: Loading Students and
their Addresses\n");
                    // Eagerly load Student entities along with their related
Address entities using method syntax.
                    var studentsWithAddressesMethod = context.Students
.Include(s => s.Address) // Eager load Address entity
using a lambda expression
                    .ToList();
                    // Method Syntax with Include (using string parameter)
                    // Eagerly load Student entities along with their related
Address entities using string-based Include.
                    //var studentsWithAddressesMethodString =
context.Students
                    // .Include("Address") // Eager load Address entity
using string parameter
                    // .ToList();
                    // Eager Loading using Query Syntax with Lambda
Expression
                    //var studentsWithAddressesQueryLambda = (from student in
context.Students
                    // .Include(s =>
s.Address) // Eagerly load Address entity using lambda in query syntax
                    // select
student).ToList();
                    // Eager Loading using Query Syntax with String
                    //var studentsWithAddressesQueryString = (from student in
context.Students
                    // .Include("Address") // Eagerly load Address entity using string in query
syntax
                    // select
student).ToList();
                    // Display results
```

```

        Console.WriteLine(); // Display a new line before
displaying the data
        foreach (var student in studentsWithAddressesMethod)
        {
            if (student.Address != null)
            {
                // Address exists, display the full address
details
                Console.WriteLine($"Student: {student.FirstName}
{student.LastName}, Address: {student.Address.Street},
{student.Address.City}, {student.Address.State}");
            }
            else
            {
                // Address is null, display "No Address"
                Console.WriteLine($"Student: {student.FirstName}
{student.LastName}, Address: No Address");
            }
        }
    }
    catch (Exception ex)
    {
        // Handle exceptions
        Console.WriteLine($"An error occurred while fetching the
data. Error: {ex.Message}");
    }
}
// Final Output
Console.WriteLine("Eager loading completed.");
}
}
}
}
}

```

Output:

Method Syntax: Loading Students and their Addresses

```

info: 08-09-2024 07:10:48.398 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database
.Command)
    Executed DbCommand (18ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
    SELECT [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[FirstName], [s].[Gender], [s].[LastName]
, [a].[AddressId], [a].[City], [a].[PostalCode], [a].[State], [a].[Street], [a].[StudentId], [a].[TeacherId]
FROM [Students] AS [s]
LEFT JOIN [Addresses] AS [a] ON [s].[StudentId] = [a].[StudentId]

Student: John Doe, Address: 123 Main St, New York, NY
Student: Jane Smith, Address: 456 Oak St, Los Angeles, CA
Student: Mark Johnson, Address: 789 Pine St, Chicago, IL
Student: Emily Clark, Address: 101 Maple St, New York, NY
Student: Sara Taylor, Address: No Address
Student: James Smith, Address: No Address
Eager loading completed.

```

As you can see in the above output, EF Core uses SQL LEFT JOIN to fetch the data with Eager Loading. Now, let us understand why EF Core uses Left Join.

When EF Core Uses a LEFT JOIN?

Entity Framework Core (EF Core) generates a LEFT JOIN when eagerly loading the Address for a Student because of the nullable relationship defined in our Address model. Specifically, the StudentId property in the Address class is marked as nullable (int?), indicating that an address may or may not be associated with a student. That means this is an optional relationship, and for an optional relationship, EF Core uses LEFT JOIN while loading the related data using Eager Loading.

So, when EF Core generates the SQL query to load the address along with the student eagerly, it uses a LEFT JOIN to ensure that even if a student does not have a corresponding address, the student will still be included in the result set. That means the LEFT JOIN ensures that all rows from the Students table are returned, even if some students have no corresponding Address. If the student does not have an address, the address fields in the result will be NULL.

What are the Differences Between Include Method with lambda and String Expression?

The Include() method in Entity Framework Core can be used with both lambda expressions and string expressions to specify related entities for eager loading, but there are key differences between the two approaches.

Lambda Expression (Include(s => s.Address)):

- Using lambda expressions provides type safety because the compiler can check the types at compile time. If there is an error, such as a misspelled property name or an incorrect type, it will be caught during compilation. Type safety ensures that the property being included (Address) is a valid navigation property on the Student entity. If you change or refactor property names, the code will break at compile time.
- Lambda expressions benefit from Intellisense in IDEs like Visual Studio. As you type, the IDE will suggest valid navigation properties based on the entity type. This feature reduces the likelihood of typos and speeds up development.

String Expression (Include("Address")):

- String expressions are not type-safe. The property name is passed as a string, and any errors, such as a typo in the property name, will only be caught at runtime, leading to exceptions.
- String expressions do not benefit from Intellisense or code completion. We must manually type the property names without assistance, making it easier to introduce errors.

Note: The Include() method with a lambda expression is recommended. In terms of how Entity Framework Core executes the query, there is no performance difference between using lambda expressions and string expressions. Both methods generate the same SQL query and have the same impact on performance.

How Do We Eager Load Multiple Related Entities Using Include Method in EF Core?

In Entity Framework Core (EF Core), eager loading multiple related entities using the Include method is a common task, especially when dealing with models that have complex relationships. The Include method allows us to load related entities along with the main entity in a single query, reducing the number of round trips to the database and ensuring that all required data is fetched together.

Let's look at an example to understand this better. Suppose we have a Student entity with several related entities, such as Branch, Address, and Courses, as shown in the image below.

```

public class Student
{
    0 references
    public int StudentId { get; set; }
    0 references
    public string FirstName { get; set; }
    0 references
    public string LastName { get; set; }
    0 references
    public string Gender { get; set; }
    0 references
    public DateTime DateOfBirth { get; set; }
    0 references
    public int BranchId { get; set; }
    0 references
    public Branch Branch { get; set; }
    0 references
    public Address Address { get; set; }
    0 references
    public ICollection<Course> Courses { get; set; }
}

```

↓ Related Entities of Student Entity

We can use the Include() method for each related entity to load all these related entities together with the Student. That means we can use the Include() method multiple times in a single query. For a better understanding, please modify the Program class as follows.

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Initialize the database context
            using (var context = new EFCoreDbContext())
            {
                try
                {
                    // METHOD SYNTAX with Include (using lambda expression)
                    Console.WriteLine("Method Syntax: Loading Students with
Branch, Address, and Courses");
                    // Eagerly load Student entities along with their related
Branch, Address, and Courses entities using method syntax.
                    var studentsWithDetailsMethod = context.Students
                        .Include(s => s.Branch) // Eagerly load
Branch entity
                        .Include(s => s.Address) // Eagerly load
Address entity

```



```

Method Syntax: Loading Students with Branch, Address, and Courses
info: 08-09-2024 07:21:26.725 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (28ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[FirstName], [s].[Gender], [s].[LastName], [b].[BranchId], [b].[BranchEmail], [b].[BranchLocation], [b].[BranchPhoneNumber], [a].[AddressId], [a].[City], [a].[PostalCode], [a].[State], [a].[Street], [a].[StudentId], [a].[TeacherId], [t].[CoursesCourseId], [t].[StudentsStudentId], [t].[CoursesCourseId], [t].[Description], [t].[Fees], [t].[Name]
      FROM [Students] AS [s]
      INNER JOIN [Branches] AS [b] ON [s].[BranchId] = [b].[BranchId]
      LEFT JOIN [Addresses] AS [a] ON [s].[StudentId] = [a].[StudentId]
      LEFT JOIN (
        SELECT [c].[CoursesCourseId], [c].[StudentsStudentId], [c0].[CourseId], [c0].[Description], [c0].[Fees], [c0].[Name]
        FROM [CourseStudent] AS [c]
        INNER JOIN [Courses] AS [c0] ON [c].[CoursesCourseId] = [c0].[CourseId]
      ) AS [t] ON [s].[StudentId] = [t].[StudentsStudentId]
      ORDER BY [s].[StudentId], [b].[BranchId], [a].[AddressId], [t].[CoursesCourseId], [t].[StudentsStudentId]

Student: John Doe, Branch: BBSR, Address: New York, Courses Count: 1
Student: Jane Smith, Branch: Delhi, Address: Los Angeles, Courses Count: 1
Student: Mark Johnson, Branch: Mumbai, Address: Chicago, Courses Count: 1
Student: Emily Clark, Branch: BBSR, Address: New York, Courses Count: 1
Student: Sara Taylor, Branch: Delhi, Address: No Address, Courses Count: 1
Student: James Smith, Branch: Mumbai, Address: No Address, Courses Count: 1

```

If you look at the above output, you will see that it uses both INNER JOIN and LEFT JOIN when generating the SQL query. We have already discussed when EF Core uses LEFT JOIN to load the related entities. Now, let us proceed and understand when it uses INNER JOIN to load the related entities.

When Does EF Core Use Inner Join in Eager Loading?

EF Core uses an INNER JOIN when the relationship between the entities is required, meaning the foreign key in the dependent entity is non-nullable. In a required relationship, EF Core assumes that the related entity always exists and can safely use an INNER JOIN to ensure only students with a matching branch are included.

When we eagerly load the Branch entity for a Student, EF Core generates an INNER JOIN because the relationship between Student and Branch is a required (non-nullable) relationship, indicated by the non-nullable foreign key (BranchId) in the Student entity. Since every student is guaranteed to have a Branch (because of the non-nullable BranchId), EF Core can safely use an INNER JOIN to retrieve the related Branch.

What is Loading Multiple Levels of Related Entities in EF Core?

In Entity Framework Core (EF Core), loading multiple levels of related entities refers to the process of loading not just the immediate related entities but also their related entities, often through nested relationships.

In EF Core, this is done using the Include() and ThenInclude() methods. These methods allow us to load multiple levels of related data in a single query, minimizing the number of database round trips. In our example, the Student entity has several relationships:

- A many-to-one relationship with a Branch (a student belongs to one branch).
- A one-to-one relationship with Address (each student has one address).
- A many-to-many relationship with Course (a student can be enrolled in multiple courses, and each course can have many students).

Additionally:

- Each Course has a many-to-many relationship with the Subject (courses can cover multiple subjects, and each subject can be part of multiple courses).
- Each Teacher can have relationships with Branch, Address, and Subject.

Loading Multiple Levels of Related Entities in EF Core:

We can use the Include() and ThenInclude() methods to load multiple levels of related entities. Let us see an example to understand this concept. Now, let us Load a Student and Their Related Branch, Address, and Courses, and also, we need to Load the related Subjects for each Course. So, please modify the Program class as follows.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Initialize the database context
            using (var context = new EFCoreDbContext())
            {
                try
                {
                    // Method Syntax with Include and ThenInclude (using
                    // lambda expressions)
                    Console.WriteLine("Loading Students and their related
                    entities\n");

                    // Eagerly load Student, Branch, Address, Courses, and
                    // the related Subjects using method syntax
                    var student = (context.Students
                        .Where(std => std.StudentId == 1)
                        .Include(s => s.Branch) // Eagerly load
                        related Branch
                        .Include(s => s.Address) // Eagerly load
                        related Address
                        .Include(s => s.Courses) // Eagerly load
                        related Courses
                        .ThenInclude(c => c.Subjects)) // Eagerly load
                        related Subjects for each Course
                        .FirstOrDefault(); // Execute the
                        query and retrieve the data

                    // Display basic student information
                    Console.WriteLine($"Student: {student.FirstName}
                    {student.LastName}");
                    Console.WriteLine($"Branch:
                    {student.Branch?.BranchLocation}");
                    Console.WriteLine($"Address: {student.Address?.Street},
                    {student.Address?.City}, {student.Address?.State}");
                    // Display each course and its related subjects
                    foreach (var course in student.Courses)
                    {
                        Console.WriteLine($"Course: {course.Name}");
                        foreach (var subject in course.Subjects)
                        {
                            Console.WriteLine($"    Subject:
                            {subject.SubjectName}");
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
}
catch (Exception ex)
{
    // Exception handling: Catch and display any errors that
    occur during data retrieval
    Console.WriteLine($"An error occurred while fetching the
data. Error: {ex.Message}");
}
}
// Final Output
Console.WriteLine("\nEager loading of related entities
completed.");
}
}
}
}

```

Output:

```

Loading Students and their related entities

Student: John Doe
Branch: BBSR
Address: 123 Main St, New York, NY
Course: Full Stack Dot Net
      Subject: ASP.NET Core Development
      Subject: C# Programming
      Subject: JavaScript Frameworks

Eager loading of related entities completed.

```

When to Use Eager Loading with EF Core:

- **You Know You'll Need Related Data Immediately:** If you know that related data (such as navigation properties) will be accessed soon after the main entity is loaded, eager loading is an excellent choice. For example, when displaying a Student along with their Branch, Courses, and Address on a page, it makes sense to eager load everything in one query to minimize round trips to the database.
- **You Want to Minimize Round Trips to the Database:** Eager loading fetches all related data in a single query, which reduces the number of queries sent to the database (i.e., no multiple queries for each related entity).
- **When You Want to Avoid the N+1 Query Problem:** The N+1 query problem occurs when a query retrieves a collection, and for each item in the collection, a separate query is executed to fetch related data. Eager loading can prevent this by loading all related data in a single query instead of triggering individual queries for each item.

In the next article, I will discuss [Lazy Loading in Entity Framework Core](#). In this article, I explain **Eager Loading in Entity Framework Core** with Examples. I hope you enjoyed this article on Eager Loading in EF Core.